Task 12: Two Pointers, Sliding Window, Mirror Images & Ratio-Proportion

12 Oct 2026 · @Snuggles

Data Structures & Algorithms — Two Pointers, Sliding Window + Mirror Images (Reasoning) + Ratio & Proportion (Aptitude) · Task 12

Task 11 mein Hashing se O(1) lookup seekha — lekin uske badle extra O(N) memory lagi. Ab agla step hai **Two Pointers** aur **Sliding Window**: do index variables ko smartly chala ke wahi kaam O(N) time aur O(1) extra space mein. Yahi LeetCode Roadmap ka Day 5–12 hai.

Akshara ke liye Two Pointers ke Easy problems + har problem ka explanation video. Hardik ke liye Sliding Window ke Medium problems (roadmap Day 9–12 + kuch extra). Saath mein Reasoning (Mirror Images) aur Aptitude (Ratio & Proportion) — is baar dono MCQ format mein, options ke saath.

1 Theory: Two Pointers

Two Pointers ka matlab: array ya string pe ek ki jagah **do index (pointers)** chalao, aur har step pe kam se kam ek pointer aage badhao. Isse nested loops ($O(N^2)$) wala kaam ek hi pass ($O(N)$) mein ho jaata hai — aur koi extra array bhi nahi chahiye ($O(1)$ space).

Analogy: Do dost ek word check kar rahe hain — ek aage se padhta hai, doosra peeche se. Har step pe dono apna letter bolte hain aur ek-ek kadam beech ki taraf badhte hain. Ek bhi letter alag hua toh palindrome nahi. Bas yahi Two Pointers hai.

Teen patterns jo baar-baar aate hain:

Pattern	Pointers kaise chalte hain	Kahan use hota hai
Opposite Ends (converging)	l = 0 aur r = n − 1 se shuru, beech mein milte hain	Palindrome, Reverse, sorted array mein pair-sum
Same Direction (read / write)	read har element dekhta hai, write batata hai agla "kaam ka" element kahan likhna hai	Remove Duplicates, Move Zeroes (in-place)
Two Arrays	Har array pe ek pointer, jo chhota ho woh aage	Merge Sorted Array — Task 9 ke Merge Sort ka merge step yahi tha!

Pehchaan kaise karein? Question mein "sorted array", "pair / triplet with sum", "palindrome", "in-place" ya "O(1) extra space" dikhe — toh pehle Two Pointers socho.

Java Templates

```java
// Pattern 1: Opposite Ends — sorted array mein pair with sum = target
static boolean hasPair(int[] arr, int target) {
    int l = 0, r = arr.length - 1;
    while (l < r) {
        int sum = arr[l] + arr[r];
        if (sum == target) return true;
        else if (sum < target) l++;   // sum chhota → bada number chahiye → left aage
        else r--;                     // sum bada → chhota number chahiye → right peeche
    }
    return false;
}

// Pattern 2: Same Direction (read / write) — sorted array se duplicates hatao
static int removeDuplicates(int[] arr) {
    if (arr.length == 0) return 0;
    int write = 1;
    for (int read = 1; read < arr.length; read++) {
        if (arr[read] != arr[read - 1]) {
            arr[write] = arr[read];
            write++;
        }
    }
    return write;   // unique elements ki count
}
```

5 Golden Rules:

1. Opposite-ends pair-sum sirf **sorted** array pe sahi chalta hai. Unsorted pe galat answer aayega aur koi error bhi nahi dikhega (Task 10 ke Binary Search jaisa).
2. Pair dhundte waqt `while (l < r)` likho, `l <= r` nahi — warna ek hi element do baar use ho jaayega.
3. Har iteration mein kam se kam ek pointer zaroor move ho — warna infinite loop.
4. Andar wale skip-loops (jaise space / comma chhodna) mein bhi `l < r` check lagao — warna index array ke bahar chala jaayega.
5. Swap hamesha `temp` se karo. `a[l] = a[r]; a[r] = a[l];` likhoge toh dono jagah ek hi value aa jaayegi (Case 1 dekho).

2 Theory: Sliding Window

Sliding Window, Two Pointers ka hi ek special case hai: dono pointers (`l` aur `r`) **ek hi direction** mein chalte hain, aur unke beech ka hissa (subarray / substring) ek "window" hai. Window ka answer har baar shuru se nahi nikalte — naya element andar aata hai (`r++`), purana bahar jaata hai (`l++`), bas utna update karo.

Analogy: Cricket mein "last 5 overs ka run rate". Naya over khatam hua toh uske runs jodo aur sabse purane over ke runs ghatao — saare 5 overs dobara nahi jodne padte.

```
arr = [2, 1, 5, 1, 3, 2],  k = 3

[2  1  5] 1  3  2      sum = 8
 2 [1  5  1] 3  2      sum = 8 - 2 + 1 = 7
 2  1 [5  1  3] 2      sum = 7 - 1 + 3 = 9   ← max
 2  1  5 [1  3  2]     sum = 9 - 5 + 2 = 6
```

Fixed vs Variable Window

	Fixed Window	Variable Window
Window ka size	Pehle se fixed (k)	Condition ke hisaab se badhta-ghatta hai
Question mein dikhega	"size k ka subarray / substring"	"longest / shortest subarray jisme …"
Pointers	r aur l saath-saath aage	r hamesha aage, l sirf tab jab window invalid ho
Examples	Permutation in String, Find All Anagrams	Longest Substring Without Repeating, Character Replacement

Kitna fast hai? "Size k ke har subarray ka max sum" — brute force har window ka sum dobara jodta hai: O(N·K). Sliding window har step pe ek add + ek subtract karti hai: O(N). N = 10 lakh aur k = 1000 ho toh brute force ~100 crore operations, window sirf ~20 lakh.

Java Templates

```java
// Fixed Window — size k ka max sum
int sum = 0, max = Integer.MIN_VALUE;
for (int r = 0; r < arr.length; r++) {
    sum += arr[r];                    // naya element andar
    if (r >= k - 1) {                 // window ka size k ho gaya
        max = Math.max(max, sum);
        sum -= arr[r - k + 1];        // sabse purana element bahar
```

```
    }
}

// Variable Window — sabse lambi "valid" window
int l = 0, best = 0;
for (int r = 0; r < n; r++) {
    // 1. arr[r] ko window mein add karo (count / sum update)
    while (/* window invalid hai */) {
        // 2. arr[l] ko window se hatao
        l++;
    }
    best = Math.max(best, r - l + 1);  // 3. window valid → answer update
}
```

Window ke andar kya-kya hai, yeh track karne ke liye `HashMap`, `HashSet` ya `int[26]` lagta hai — Task 11 ka frequency counting yahin kaam aayega.

5 Golden Rules:

1. Window ki length = `r - l + 1` (dono ends included). Substring nikalni ho toh `s.substring(l, r + 1)` — end exclusive hota hai.
2. Variable window mein shrink ke liye `while` likho, `if` nahi — kabhi ek saath kai elements nikalne padte hain (Case 8).
3. Negative numbers ho sakte hain toh `max` ko `0` se nahi, `Integer.MIN_VALUE` se shuru karo (Case 7).
4. Sum wali sliding window tab hi chalti hai jab saare numbers non-negative hon. Negative aaye toh prefix sum + HashMap (Task 11 ka LC 560 yaad hai?).
5. Window se element nikalte waqt map mein count ghatao, aur 0 ho jaaye toh key `remove()` karo — warna `map.size()` galat aayega.

3 DSA Practice Problems — Akshara ke liye (Two Pointers · Easy)

Hashing → Two Pointers. Saare 6 problems Easy hain. Har problem mein pehle brute force socho (extra array ya nested loop), phir two pointers se O(N) time aur O(1) extra space mein karo. Problems se pehle concept videos dekho; har problem ke saath uska explanation video bhi diya hai — pehle khud try karo, atak jao tab video dekho.

Concept videos / reading:

#	Resource	Kya dekhna hai

1	Striver (take U forward) — L1. Introduction to Sliding Window and 2 Pointers	Is task ka main video. Two pointers + templates wala part zaroor; sliding window wala part bonus.
2	GeeksforGeeks — Two Pointers Technique	Pair-sum ka illustration — l++ / r-- kab karna hai, step-by-step.
3	Task 9 ka Merge Sort (merge step)	Revision — Q6 (Merge Sorted Array) wahi idea hai.

1. Reverse String · LeetCode 344

Character array ko in-place ulta karo: ["h","e","l","l","o"] → ["o","l","l","e","h"]. Naya array banana allowed nahi.

Hint: `l = 0`, `r = n - 1`. Jab tak `l < r`: `temp` se swap karo, phir `l++`, `r--`. Task 10 ka "Reverse the Number" yaad hai? Wahi idea, ab array pe. (Case 1 dekho — temp ke bina kya hota hai.)

→ leetcode.com/problems/reverse-string · ▶ Video: Reverse String — Java solution (Teddy Smith)

2. Valid Palindrome · LeetCode 125 · Roadmap Day 5

Capital ko small banao aur letters/digits ke alawa sab ignore karo — phir check karo ki string aage-peeche se same hai. "A man, a plan, a canal: Panama" → true, "race a car" → false.

Hint: Do pointers dono ends se. `Character.isLetterOrDigit()` false ho toh us pointer ko aage/peeche khiskao (inner loop mein bhi `l < r`!). Compare karte waqt `Character.toLowerCase()` lagao. Yahi check Reasoning ke Mirror Words wale questions mein bhi kaam aayega.

→ leetcode.com/problems/valid-palindrome · ▶ Video: Valid Palindrome (NeetCode)

3. Remove Duplicates from Sorted Array · LeetCode 26

Sorted array se duplicates in-place hatao aur unique elements ki count `k` return karo. [0,0,1,1,1,2,2,3,3,4] → k = 5, array ke pehle 5 elements [0,1,2,3,4].

Hint: Read / write pattern (Theory ka template 2). `read` har element dekhta hai; naya value mile toh `arr[write] = arr[read]` aur `write++`. Sorted hai, isliye duplicates hamesha saath-saath honge.

→ leetcode.com/problems/remove-duplicates-from-sorted-array · ▶ Video: Remove Duplicates — Hindi explanation

4. Move Zeroes · LeetCode 283

Saare 0 ko end mein bhejo, baaki elements ka order same rehna chahiye. [0,1,0,3,12] → [1,3,12,0,0].

Hint: Q3 jaisa hi read / write. Non-zero mila toh `write` position pe likho. Loop ke baad `write` se end tak 0 bharna mat bhoolna (Case 5 dekho) — ya seedha swap karo.

→ leetcode.com/problems/move-zeroes · ▶ Video: Move Zeroes — Python, JS, Java, C++

5. Squares of a Sorted Array · LeetCode 977

Sorted array (negative bhi ho sakte hain) ke har element ka square nikalo, result bhi sorted chahiye. [-4,-1,0,3,10] → [0,1,9,16,100].

Hint: Pehle brute force: square karo + `Arrays.sort()` = O(N log N). Phir socho — sabse bada square hamesha kisi ek end pe hoga (-4 ya 10). Dono ends compare karo aur result array ko peeche se bharo → O(N).

→ leetcode.com/problems/squares-of-a-sorted-array · ▶ Video: Squares of a Sorted Array — Java, Two Pointers

6. Merge Sorted Array · LeetCode 88

`nums1` (size m + n, last n jagah khaali) aur `nums2` (size n) — dono sorted. `nums2` ko `nums1` mein merge karo, result sorted ho.

Hint: Teen pointers: `i = m - 1`, `j = n - 1`, `k = m + n - 1`. **Peeche se** bharo — jo bada ho woh `nums1[k]` mein. Aage se bharoge toh `nums1` ke elements overwrite ho jaayenge.

→ leetcode.com/problems/merge-sorted-array · ▶ Video: Merge Sorted Array (NeetCode)

Bonus — Two Sum II: Input Array Is Sorted · LeetCode 167 · Roadmap Day 5

Task 11 ka Two Sum, bas array sorted hai aur extra space O(1) chahiye — HashMap nahi chalega. LeetCode pe Medium tag hai, lekin logic seedha Theory ka template 1 hai. Answer 1-indexed return karna hai.

→ leetcode.com/problems/two-sum-ii-input-array-is-sorted · ▶ Video: Two Sum II (NeetCode)

4 DSA Practice Problems — Hardik ke liye (Sliding Window · Medium)

LeetCode Roadmap Day 9–12 (Sliding Window) + 2 extra Medium jo interview mein bahut aate hain. Har problem ke liye pehle brute force ki complexity likho, phir O(N) window. Variable window ka template (Theory section 2) har problem pe fit karke dikhao.

Concept video: Striver — L1. Introduction to Sliding Window and 2 Pointers — poora dekho, khaas kar 4 patterns wala part (constant window, longest window, count of subarrays, shortest window).

Warm-up (Easy, 10–15 min) — Best Time to Buy and Sell Stock · LeetCode 121 · Roadmap Day 9

Ek din khareedo, baad ke kisi din becho — max profit. `l` = sabse sasta din ab tak, `r` aage badhta hai. Seedha O(N), ek pass.

→ leetcode.com/problems/best-time-to-buy-and-sell-stock · ▶ Video: Sliding window explanation · NeetCode written solution (Java bhi)

1. Longest Substring Without Repeating Characters · LeetCode 3 · Roadmap Day 9

Sabse lambi substring jisme koi character repeat na ho. "abcabcbb" → 3 ("abc").

Hint: Window ke characters `HashSet` mein. `s[r]` pehle se set mein hai toh `while` loop se `s[l]` hatao aur `l++` (Case 8 — `if` kyun fail hota hai). Optimization: `HashMap<Character, Integer>` mein last index rakho aur `l = Math.max(l, last + 1)` se seedha jump karo. README mein likho: yahan `Math.max` kyun zaroori hai? ("abba" pe dry run karo.)

→ leetcode.com/problems/longest-substring-without-repeating-characters · ▶ Video: NeetCode

2. Longest Repeating Character Replacement · LeetCode 424 · Roadmap Day 10

String mein max `k` characters badal sakte ho. Sabse lambi substring jisme saare letters same ho jaayein? "AABABBA", k = 1 → 4.

Hint: Window valid hai jab `(windowLength - maxFreq) <= k` — yaani jo letters sabse common letter nahi hain, wahi badalne padenge. `int[26]` count rakho. README mein 2 lines: window shrink karte waqt `maxFreq` ko kam kyun nahi karna padta?

→ leetcode.com/problems/longest-repeating-character-replacement · ▶ Video: Java solution (Teddy Smith)

3. Permutation in String · LeetCode 567 · Roadmap Day 10

Kya `s2` mein `s1` ka koi permutation (anagram) substring ke roop mein hai? s1 = "ab", s2 = "eidbaooo" → true ("ba").

Hint: **Fixed window**, size = `s1.length()`. Do `int[26]` count arrays — window slide karte waqt ek letter jodo, ek ghatao. Compare `Arrays.equals()` se karo, `==` se nahi (Case 10). Upgrade: ek `matches` counter (0–26) rakho taaki har step pe 26 compare na karne padein.

→ leetcode.com/problems/permutation-in-string · ▶ Video: Sliding window in Java, 2 approaches

4. Find All Anagrams in a String · LeetCode 438 · Roadmap Day 11

Q3 ka extension — true / false ki jagah `p` ke har anagram ka starting index return karo. s = "cbaebabacd", p = "abc" → [0, 6].

Hint: Q3 ka code almost as-is reuse karo — match mile toh `return true` ki jagah `l` ko list mein add karo. Dono files ka diff README mein dikhao.

→ leetcode.com/problems/find-all-anagrams-in-a-string · Explanation: NeetCode written solution (brute → optimal, Java)

5. Max Consecutive Ones III · LeetCode 1004 · Extra

Binary array mein max `k` zeros ko 1 bana sakte ho. Sabse lambi 1s ki line? [1,1,1,0,0,0,1,1,1,1,0], k = 2 → 6.

Hint: Window mein zeros ki count `<= k` rakho. Yeh Q2 (424) ka hi chhota bhai hai — sirf do "letters" (0 aur 1). Dono ka code side-by-side compare karo.

→ leetcode.com/problems/max-consecutive-ones-iii · ▶ Video: Striver L4

6. Minimum Size Subarray Sum · LeetCode 209 · Extra

Positive numbers ka array. Sabse **chhota** subarray jiska sum `>= target` ho, uski length (na ho toh 0). target = 7, [2,3,1,2,4,3] → 2.

Hint: Ab "longest" nahi, "shortest" chahiye — isliye answer `while` loop ke **andar** update hoga (window valid rehte hue jitna chhota kar sako). Task 11 ke LC 560 se compare karo: wahan negative numbers the, isliye window nahi chali thi.

→ leetcode.com/problems/minimum-size-subarray-sum · Explanation: NeetCode written solution (Java bhi)

Bonus (Hard, optional) — Roadmap Day 11–12

- **Minimum Window Substring · LeetCode 76** — `s` ka sabse chhota window jisme `t` ke saare characters (duplicates samet) hon. `have` / `need` counters + Q6 jaisi shrinking. → leetcode.com/problems/minimum-window-substring · ▶ Video: Striver L12
- **Sliding Window Maximum · LeetCode 239** — har size-k window ka max. Heap se O(N log K) pehle likho, phir monotonic `ArrayDeque` (indices store karo) se O(N). → leetcode.com/problems/sliding-window-maximum · ▶ Video: Deque approach (Nikhil Lohia)

Roadmap tracker update: `LeetCode_150_Roadmap.xlsx` mein Day 9–11 ke 5 rows tick karo — Best Time to Buy and Sell Stock, Longest Substring Without Repeating Characters, Longest Repeating Character Replacement, Permutation in String, Find All

Anagrams in a String. Minimum Window Substring aur Sliding Window Maximum sirf tab tick karna jab bonus submit ho. Day 5–8 (Two Pointers) ke rows agle task mein aayenge; time bache toh 3Sum (LC 15) aur Container With Most Water (LC 11) abhi kar lo.

5 Reasoning: Mirror Images (Akshara ke liye)

Mirror Images mein dekhna hota hai ki koi letter, word, number, clock ya insaan mirror mein kaisa dikhega. TCS NQT, SSC, bank aur railway exams — sab mein aata hai, aur yeh 30 second ka sawaal hota hai agar rules pakke hon. Aur maza yeh hai ki "word mirror mein same dikhega ya nahi" — yeh literally Two Pointers wala palindrome check hai!

Do tarah ke mirror:

	Vertical mirror (mirror right / left mein)	Water image (mirror neeche)
Kya ulta hota hai	Left ↔ Right (lateral inversion)	Upar ↔ Neeche
Word ke letters ka order	Ulta ho jaata hai (aakhri letter pehle)	Same rehta hai
Kaunse letters same dikhte hain	A H I M O T U V W X Y	B C D E H I K O X
Dono mein same	H I O X	H I O X

Sochne ka tarika:

1. Pehle pucho — mirror kahan hai? Question mein "mirror" likha ho aur kuch na bataya ho, toh vertical mirror (right side) maano. "Water image" ka matlab hamesha mirror neeche.
2. **Vertical mirror mein word:** letters ka order ulta + har letter left-right flip. Agar saare letters symmetric hain (A H I M O T U V W X Y), toh image bas reverse string jaisi padhi jaayegi.
3. **Word bilkul same dikhe** iske liye do shart: word palindrome ho **aur** har letter vertically symmetric ho. Palindrome check = Two Pointers (Akshara ka Q2)!
4. **Clock — mirror time = 11:60 − actual time.** Jaise 3:40 → 11:60 − 3:40 = 8:20. Time 12:xx ho toh 23:60 se ghatao (12:20 → 11:40). Seconds ho toh 11:59:60 se ghatao.
5. **Mirror ka mirror = original.** Isliye "mirror mein 4:35 dikh raha hai, asli time kya hai?" ka formula bhi wahi hai.
6. **Insaan aur mirror:** right hand uthao toh image left hand uthati dikhti hai. Lekin mirror sirf woh direction ulta karta hai jo uski taraf hai (aage ↔ peeche) — East ki taraf point karoge toh image bhi East ki taraf hi point karegi.

7. **Real-life check:** Ambulance pe "AMBULANCE" ulta likha hota hai, taaki aage wali gaadi ke driver ko rear-view mirror mein seedha padhne mein aaye.

15 Practice Questions (MCQ)

Har question ke liye `MirrorImages.md` mein sahi option + 1–2 line reason likhna hai (clock waale mein subtraction steps bhi).

LEVEL 1 — LETTER SYMMETRY

Q1. Which group contains only letters that look exactly the same in a vertical mirror (mirror placed to the right)? (a) A, H, M, T (b) B, D, E, K (c) A, B, M, N (d) F, J, P, R

Q2. Which group contains only letters that look exactly the same in their water image (mirror placed below)? (a) B, C, D, E (b) A, M, T, V (c) B, C, N, S (d) F, G, L, P

Q3. Which group contains only letters that look the same in BOTH a vertical mirror and a water image? (a) H, I, O, X (b) A, M, U, W (c) B, C, D, K (d) H, M, O, T

LEVEL 2 — WORDS IN A MIRROR

Q4. Which word looks exactly the same when seen in a vertical mirror? (a) NOON (b) MOM (c) DAD (d) EYE

Q5. Which word looks exactly the same in its water image? (a) CHECKBOOK (b) KNOWLEDGE (c) BRIDGE (d) MOTHER

Q6. Every letter of the word TOMATO is vertically symmetric. Reading left to right, how will its image in a vertical mirror read? (a) TOMATO (b) OTAMOT (c) OTMATO (d) TAMOTO

Q7. When the word ACADEMY is seen in a vertical mirror, how many of its letters (count every occurrence) will look exactly the same in shape? (a) 2 (b) 3 (c) 4 (d) 5

Q8. A word looks exactly the same when seen in a vertical mirror. Which statement MUST be true? (a) It is a palindrome AND every letter is vertically symmetric (b) It only needs to be a palindrome (c) It only needs vertically symmetric letters (d) It must have an even number of letters

LEVEL 3 — CLOCKS

Q9. A wall clock shows 3:40. What time will its mirror image show? (a) 8:20 (b) 9:20 (c) 8:40 (d) 7:20

Q10. In a mirror, a clock appears to show 4:35. What is the actual time? (a) 6:25 (b) 7:25 (c) 7:35 (d) 8:25

Q11. What time will the mirror image of a clock show when the actual time is 12:20? (a) 12:40 (b) 11:20 (c) 11:40 (d) 10:40

Q12. A clock with a seconds hand shows 2:47:30. What time will its mirror image show? (a) 9:12:30 (b) 9:13:30 (c) 10:12:30 (d) 9:12:20

Q13. At which times does a clock look exactly the same as its own mirror image? (a) 3:00 and 9:00 (b) 6:00 and 12:00 (c) 12:00 only (d) 3:00 only

LEVEL 4 — PEOPLE & DIRECTIONS

Q14. Riya stands facing a mirror and raises her right hand. Her image appears to raise which hand? (a) Right hand (b) Left hand (c) Both hands (d) Cannot be determined

Q15. Aman faces North, looking into a mirror fixed on the wall in front of him. He points his right arm towards the East. Towards which direction does his image's arm point? (Task 11 ka Direction Sense yaad karo.) (a) West (b) East (c) North (d) South

Aur practice chahiye?

→ IndiaBix: Mirror Images — picture-based questions (figures + letter/number combinations). Yahan ke text questions ke baad yeh zaroor karna, kyunki exam mein mostly figures aate hain.

6 Aptitude: Ratio & Proportion (Akshara ke liye)

Ratio do quantities ki tulna hai (a : b = a / b), aur Proportion ka matlab do ratios barabar hain (a : b = c : d). TCS NQT, Infosys, Wipro, AMCAT aur bank exams mein har saal 2–3 sawaal isi se aate hain — aur Partnership, Ages, Mixture jaise topics bhi isi pe khade hain.

Analogy: Chai ki recipe — 2 cup doodh : 1 cup paani. 4 log aaye toh 4 : 2, 10 log aaye toh 10 : 5. Quantity badli, ratio wahi raha. Isliye ratio ko hamesha `2x : 1x` likho — `x` hi asli quantity batata hai.

Formula Cheat Sheet

Concept	Formula / Trick	Example
Ratios jodna (A:B aur B:C)	Common term B ko LCM se barabar karo	2:3 aur 4:5 → 8:12 aur 12:15 → 8:12:15
Amount baantna	A ka hissa = Total × a / (a + b + c)	₹900 in 2:3:4 → A = 900 × 2/9 = ₹200
Fractional ratio	Saare denominators ke LCM se multiply karo	1/2 : 1/3 = 3 : 2 (× 6)
Proportion	a : b = c : d ⇔ a × d = b × c	2 : 3 = 6 : 9 → 2 × 9 = 3 × 6
Fourth proportional (a, b, c)	d = b × c / a	2, 3, 6 → 9
Third proportional (a, b)	b^2 / a	4, 6 → 36/4 = 9

Mean proportional (a, b)	√(a × b)	4, 9 → 6
Componendo-Dividendo	a/b = c/d ⇒ (a + b)/(a − b) = (c + d)/(c − d) — ulta bhi chalta hai	(x + y)/(x − y) = 3 ⇒ x/y = 4/2 = 2
Partnership	Profit ∝ Capital × Time (months)	₹10k × 12 vs ₹20k × 6 → 1 : 1
Income–Expenditure	Savings = Income − Expenditure	Income 5x, Exp 3y

5 Golden Tricks:

1. Ratio ko hamesha `ax : bx` likho aur `x` nikalo — 90% sawaal ek line ki equation ban jaate hain.
2. "Har number mein k jodo/ghatao toh naya ratio …" — equation se pehle options daal ke check karo. Exam mein 20 second bachenge.
3. Ratio mein unit same honi chahiye — ₹ aur paise, ya ghante aur minute, mix mat karo.
4. Coins ke sawaal mein ratio **number** ka hai ya **value** ka — dhyaan se padho. Number ratio ho toh har coin ki value se multiply karo.
5. Partnership mein time bhool gaye toh galat answer pakka — jo baad mein join kare, uske months kam hain.

15 Medium Practice Questions (MCQ)

Har question ke liye `RatioProportion.md` mein sahi option + poori working likhni hai — sirf option nahi chalega.

LEVEL 1 — BASICS

Q1. If A : B = 2 : 3 and B : C = 4 : 5, then A : B : C is: (a) 8 : 12 : 15 (b) 2 : 3 : 5 (c) 8 : 12 : 5 (d) 6 : 12 : 15

Q2. The fourth proportional to 4, 9 and 12 is: (a) 27 (b) 24 (c) 36 (d) 18

Q3. The mean proportional between 12 and 75 is: (a) 30 (b) 40 (c) 45 (d) 36

Q4. The third proportional to 16 and 36 is: (a) 81 (b) 72 (c) 64 (d) 96

Q5. ₹3,900 is divided among A, B and C in the ratio 1/2 : 1/3 : 1/4. What is B's share? (a) ₹1,200 (b) ₹1,300 (c) ₹900 (d) ₹1,800

LEVEL 2 — CHANGING RATIOS

Q6. Two numbers are in the ratio 3 : 5. If 9 is subtracted from each, the ratio becomes 12 : 23. What is the smaller number? (a) 33 (b) 55 (c) 27 (d) 39

Q7. What number must be added to each of 6, 15, 20 and 43 so that they become proportional? (a) 3 (b) 2 (c) 4 (d) 1

Q8. The present ages of Ravi and Sita are in the ratio 4 : 3. After 6 years, the ratio will be 11 : 9. What is Ravi's present age? (a) 16 years (b) 12 years (c) 20 years (d) 24 years

Q9. The monthly salaries of A and B are in the ratio 3 : 5. If each gets an increment of ₹2,000, the ratio becomes 7 : 11. What is B's present salary? (a) ₹20,000 (b) ₹12,000 (c) ₹22,000 (d) ₹18,000

Q10. A 45-litre mixture contains milk and water in the ratio 7 : 2. How much water must be added to make the ratio 5 : 2? (a) 4 litres (b) 5 litres (c) 3 litres (d) 6 litres

LEVEL 3 — APPLICATIONS

Q11. A bag contains ₹1, 50-paise and 25-paise coins in the ratio 5 : 6 : 8 by number. If the total value is ₹210, how many 25-paise coins are there? (a) 168 (b) 126 (c) 105 (d) 160

Q12. If a : b = 3 : 4, then (5a − 3b) : (7a + 2b) is: (a) 3 : 29 (b) 1 : 9 (c) 3 : 23 (d) 5 : 29

Q13. If (x + y) : (x − y) = 7 : 3, then x : y is: (a) 5 : 2 (b) 2 : 5 (c) 7 : 3 (d) 10 : 3

Q14. A starts a business with ₹40,000. After 4 months, B joins with ₹60,000. At the end of the year the profit is ₹36,000. What is B's share? (a) ₹18,000 (b) ₹21,600 (c) ₹14,400 (d) ₹24,000

Q15. The incomes of A and B are in the ratio 5 : 4 and their expenditures are in the ratio 3 : 2. If each of them saves ₹1,600 a month, what is A's monthly income? (a) ₹4,000 (b) ₹3,200 (c) ₹5,000 (d) ₹4,800

Aur practice chahiye?

→ IndiaBix: Ratio and Proportion — formulas page + solved general questions with explanations.

7 Tricky Code Analysis (10 Questions)

Dono ke liye. In codes ko paper pe dry run karo — har iteration ke baad `l`, `r` (aur window ka sum / set) likho. Output predict karo, ya agar runtime error hai toh reason batao. Aur har case ke liye ek line mein fix bhi likho.

Case 1 — Swap Without temp

```
public class Test1 {
    public static void main(String[] args) {
        char[] s = {'h', 'e', 'l', 'l', 'o'};
        int l = 0, r = s.length - 1;
        while (l < r) {
            s[l] = s[r];
            s[r] = s[l];
            l++;
```

```
            r--;
        }
        System.out.println(new String(s));
    }
}
```

Case 2 — Skipping Characters Without a Bound Check

```
public class Test2 {
    public static void main(String[] args) {
        String s = ".,";
        int l = 0, r = s.length() - 1;
        while (l < r) {
            while (!Character.isLetterOrDigit(s.charAt(l))) l++;
            while (!Character.isLetterOrDigit(s.charAt(r))) r--;
            if (Character.toLowerCase(s.charAt(l)) !=
Character.toLowerCase(s.charAt(r))) {
                System.out.println(false);
                return;
            }
            l++;
            r--;
        }
        System.out.println(true);
    }
}
```

Case 3 — Pair Sum on an Unsorted Array

```
public class Test3 {
    public static void main(String[] args) {
        int[] arr = {8, 1, 5, 3};
        int target = 8;
        int l = 0, r = arr.length - 1;
        while (l < r) {
            int sum = arr[l] + arr[r];
            if (sum == target) {
                System.out.println(arr[l] + " + " + arr[r]);
                return;
            } else if (sum < target) l++;
            else r--;
        }
        System.out.println("Not found");
    }
}
```

Case 4 — `l <= r` in Pair Sum

```
public class Test4 {
    public static void main(String[] args) {
        int[] arr = {2, 5, 7};
        int target = 10;
        int l = 0, r = arr.length - 1;
        while (l <= r) {
            int sum = arr[l] + arr[r];
            if (sum == target) {
                System.out.println(l + " " + r);
                return;
            } else if (sum < target) l++;
            else r--;
        }
        System.out.println("Not found");
    }
}
```

Case 5 — Move Zeroes, Half Done

```
import java.util.Arrays;

public class Test5 {
    public static void main(String[] args) {
        int[] a = {0, 1, 0, 3, 12};
        int write = 0;
        for (int read = 0; read < a.length; read++) {
            if (a[read] != 0) {
                a[write] = a[read];
                write++;
            }
        }
        System.out.println(Arrays.toString(a));
        System.out.println(write);
    }
}
```

Case 6 — Sort Colors (Dutch National Flag) with One Extra `mid++`

```
import java.util.Arrays;

public class Test6 {
    static void swap(int[] a, int i, int j) {
        int t = a[i]; a[i] = a[j]; a[j] = t;
    }
```

```
    public static void main(String[] args) {
        int[] a = {1, 2, 0};
        int low = 0, mid = 0, high = a.length - 1;
        while (mid <= high) {
            if (a[mid] == 0) {
                swap(a, low, mid);
                low++; mid++;
            } else if (a[mid] == 1) {
                mid++;
            } else {
                swap(a, mid, high);
                high--; mid++;
            }
        }
        System.out.println(Arrays.toString(a));
    }
}
```

Case 7 — Window Max That Starts From 0

```
public class Test7 {
    public static void main(String[] args) {
        int[] a = {-4, -2, -7, -1};
        int k = 2, sum = 0, max = 0;
        for (int r = 0; r < a.length; r++) {
            sum += a[r];
            if (r >= k - 1) {
                max = Math.max(max, sum);
                sum -= a[r - k + 1];
            }
        }
        System.out.println(max);
    }
}
```

Case 8 — Shrinking With `if` Instead of `while`

```
import java.util.HashSet;

public class Test8 {
    public static void main(String[] args) {
        String s = "abba";
        HashSet<Character> set = new HashSet<>();
        int l = 0, best = 0;
        for (int r = 0; r < s.length(); r++) {
            if (set.contains(s.charAt(r))) {
```

```
                set.remove(s.charAt(l));
                l++;
            }
            set.add(s.charAt(r));
            best = Math.max(best, r - l + 1);
        }
        System.out.println(best);
    }
}
```

Case 9 — `substring()` and Window Length

```
public class Test9 {
    public static void main(String[] args) {
        String s = "racecar";
        int l = 1, r = 5;
        System.out.println(s.substring(l, r));
        System.out.println(s.substring(l, r + 1));
        System.out.println(r - l + 1);
        System.out.println(s.substring(r, l));
    }
}
```

Case 10 — Comparing Two Count Arrays

```
import java.util.Arrays;

public class Test10 {
    public static void main(String[] args) {
        int[] c1 = new int[26];
        int[] c2 = new int[26];
        for (char ch : "listen".toCharArray()) c1[ch - 'a']++;
        for (char ch : "silent".toCharArray()) c2[ch - 'a']++;
        System.out.println(c1 == c2);
        System.out.println(c1.equals(c2));
        System.out.println(Arrays.equals(c1, c2));
    }
}
```

8 Code Modification Challenge

Neeche ek program hai jo do kaam nested loops se karta hai: (A) sorted array mein saare pairs jinka sum = target, aur (B) kisi bhi 3 consecutive elements ka max sum. Dono $O(N^2)$ / $O(N \cdot K)$ hain — inhe Two Pointers aur Sliding Window se $O(N)$ banana hai.

PairsAndWindows.java — Starting Point

```java
public class PairsAndWindows {
    public static void main(String[] args) {
        // Part A: sorted array mein saare pairs jinka sum = target
        int[] sorted = {1, 3, 4, 5, 7, 9, 10, 12};
        int target = 13;
        for (int i = 0; i < sorted.length; i++) {
            for (int j = i + 1; j < sorted.length; j++) {
                if (sorted[i] + sorted[j] == target) {
                    System.out.println(sorted[i] + " + " + sorted[j]);
                }
            }
        }

        // Part B: kisi bhi 3 consecutive elements ka max sum
        int[] marks = {4, 2, 12, 1, 7, 8, 3, 9};
        int k = 3;
        int max = Integer.MIN_VALUE;
        for (int i = 0; i + k <= marks.length; i++) {
            int sum = 0;
            for (int j = i; j < i + k; j++) {
                sum += marks[j];
            }
            max = Math.max(max, sum);
        }
        System.out.println("Max window sum: " + max);
    }
}
```

Tumhara Task — Is code ko modify karo taaki yeh:

1. **Part A** ko Two Pointers (opposite ends) se O(N) mein kare — wahi 3 pairs, chhota number pehle (1 + 12, 3 + 10, 4 + 9).
2. Part A **duplicates** bhi sambhaale: `{1, 1, 2, 3, 4, 4}` aur target = 5 pe sirf unique pairs print hon — `1 + 4` aur `2 + 3`. (Hint: match milne ke baad dono pointers ko same values ke aage skip karwao. Brute force yahan `1 + 4` chaar baar print karta hai!)
3. **Part B** ko Fixed Sliding Window se O(N) mein kare, aur max sum ke saath window ka starting index aur elements bhi print kare. Tie ho toh pehli window: `Max window sum: 20 (index 2: [12, 1, 7])`.
4. **Part C (naya)** — Variable window: `marks` mein sabse chhota subarray jiska sum ≥ 15 ho, uski length aur elements print kare (Hardik ka LC 209 yahi hai).
5. Dono parts mein loop kitni baar chala (steps), count karke print kare — nested loops vs Two Pointers, aur nested loops vs Sliding Window.

Bonus (optional) — MirrorChecker.java: Reasoning aur code ko jodo.

- `isMirrorWord(String word)` — capital letters ka word vertical mirror mein bilkul same dikhega ya nahi. Two pointers se palindrome check + har letter `"AHIMOTUVWXY"` mein hona chahiye. Reasoning Q4 aur Q8 verify karo.
- `isWaterImageSame(String word)` — har letter `"BCDEHIKOX"` mein ho (yahan order nahi badalta, toh palindrome zaroori nahi). Q5 verify karo.
- `mirrorTime(int h, int m)` — mirror clock ka time return kare (12:xx wala case bhi). Hint: total minutes = (h % 12) × 60 + m, mirror = (720 − total) % 720. Q9–Q11 aur Q13 verify karo.

9 Submission & Deployment

Dono students ko apna code `task_12/` folder mein push karke GitHub link submit karni hai.

```
task_12/
├── Akshara/
│   ├── ReverseString.java
│   ├── ValidPalindrome.java
│   ├── RemoveDuplicates.java
│   ├── MoveZeroes.java
│   ├── SquaresOfSortedArray.java
│   ├── MergeSortedArray.java
│   ├── TwoSumII.java                (bonus)
│   ├── MirrorImages.md              (15 answers + har ek ka reason)
│   └── RatioProportion.md           (15 answers + poori working)
├── Hardik/
│   ├── BestTimeToBuySell.java
│   ├── LongestSubstringNoRepeat.java
│   ├── CharacterReplacement.java
│   ├── PermutationInString.java
│   ├── FindAllAnagrams.java
│   ├── MaxConsecutiveOnesIII.java
│   ├── MinSizeSubarraySum.java
│   ├── MinWindowSubstring.java      (bonus)
│   ├── SlidingWindowMaximum.java    (bonus)
│   └── README.md                    (har problem ki time/space complexity +
LC 3, 424, 438 wale answers)
├── TrickyCodeAnalysis.md            (10 cases ke output + reasoning + fix)
├── PairsAndWindows.java             (modified — two pointers + duplicates +
window + Part C + step count)
└── MirrorChecker.java               (bonus — optional)
```